

Document Number: P3234R1
Date: 2024-04-16
Project: Programming Language C++
Audience: LEWG, EWG
Author: Glen Joseph Fernandes (glenjofe@gmail.com)

Utility to check if a pointer is in a given range

This paper proposes adding a new function template `pointer_in_range`, a utility that can check if a pointer is in a given range, and can be used in a constant expression.

Changes in Revision 1

Added rationale for the choice of two pointer parameters instead of a single range parameter.

Motivation

Library authors often need this functionality and implement it themselves. A solution in the standard library would be more convenient, portable, optimal, and correct.

Common usage

A common use is determining if string ranges overlap, to be able to use a fast copy operation:

```
if (!pointer_in_range(ptr, data_, data_ + size_)) {  
    std::copy(ptr, ptr + size, data_);  
}
```

Another use is allocators that first use automatic storage, falling back to dynamic allocation:

```
if (!pointer_in_range(ptr, store_, store_ + size_)) {  
    ::operator delete(ptr);  
}
```

Existing practice

This function appears in some form in many projects, including large library collections such as:

- libc++: `__is_pointer_in_range`
- Qt: `q_points_into_range`
- Boost: `pointer_in_range`, `ptr_in_range`

Other libraries contain the same check inline even if they do not define a function for it, such as:

- libstdc++: `basic_string::_M_disjunct`
- BSL: `basic_string::privateReplaceRaw`
- EASTL: `basic_string::replace`

Imperfect solutions

Users can not implement this perfectly. For example, the following solution is not correct when the built-in operators for pointers do not yield a strict total order:

```
template<class T>
bool pointer_in_range(T* ptr, T* begin, T* end)
{
    return begin <= ptr && ptr < end;
}
```

The following solution uses comparisons consistent with the implementation-defined total order but is still not correct when two arrays of T may be interleaved:

```
template<class T>
bool pointer_in_range(T* ptr, T* begin, T* end)
{
    return std::less_equal<>()(begin, ptr) && std::less<>()(ptr, end);
}
```

Neither function above can be used in constant expressions. The following solution is correct and usable in a constant expression but is not optimal at runtime:

```
template<class T>
constexpr bool pointer_in_range(T* ptr, T* begin, T* end)
{
    for (; begin != end; ++begin) {
        if (begin == ptr) {
            return true;
        }
    }
    return false;
}
```

Argument order

The argument order (ptr, begin, end) is consistent with English and mathematical notation:

- the pointer ptr is in the range [begin, end)
- $\text{ptr} \in [\text{begin}, \text{end})$

This order is also consistent with other standard library names that are read left to right:

- `is_base_of_v<Base, Derived>`
- `is_convertible_v<From, To>`
- `less<T>(lhs, rhs)`

Why not span?

Span's convenience is also a double edged sword. Now it can even be implicitly constructed from an initializer list of two elements which allows:

```
if (!pointer_in_range(p, {x, y})) {
    // always true
}
```

This function will typically use an intrinsic that operates on raw pointers and thus its interface should least inhibit the analyzer or the optimizer:

```
template<class T>
constexpr bool pointer_in_range(const T* ptr, const T* begin, const T* end)
{
    return __builtin_pointer_in_range(ptr, begin, end);
}
```

Header choice

The header <memory> is the home of other functionality for dealing with pointers such as align and to_address.

Implementation

The [Boost](#) C++ library collection now also has the following implementation in the Core library, releasing in version 1.86, for supported platforms.

```
template<class T>
constexpr bool pointer_in_range(const T* ptr, const T* begin, const T* end)
{
    if (std::is_constant_evaluated()) {
        for (; begin != end; ++begin) {
            if (begin == ptr) {
                return true;
            }
        }
        return false;
    }
    return std::less_equal<>()(begin, ptr) && std::less<>()(ptr, end);
}
```

Limitations

At runtime, this targets only the platforms that Boost supports, which does not include implementations where two arrays of T may be interleaved.

Proposed Wording

All changes are relative to [N4971](#).

1. Insert into 20.2.2 [memory.syn] as follows:

```
// [pointer.conversion], pointer conversion
template <class T> constexpr T* to_address(T* p) noexcept;
template <class Ptr> constexpr auto to_address(const Ptr& p) noexcept;

// [pointer.range.check], pointer range check
template <class T> constexpr bool pointer_in_range(const T* ptr, const T* begin,
const T* end);
```

2. Insert after 20.2.4 [pointer.conversion] as follows:

20.2.5 Pointer range check [pointer.range.check]

```
template <class T> constexpr bool pointer_in_range(const T* ptr, const T* begin,
const T* end);
```

Mandates: T is not a function type.

Preconditions: end is reachable from begin.

Returns: As if:

```
    for (; begin != end; ++begin) {
        if (begin == ptr) {
            return true;
        }
    }
    return false;
```

Recommended practice: Implementations should be O(1) on platforms when possible.

Acknowledgments

Peter Dimov and Jens Maurer provided feedback that improved the first revision of this paper. Peter Dimov also reviewed the Boost implementation.

References

- N4971, Working Draft, Standard for Programming Language C++,
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4971.pdf>